Signal catalog

Every signal this system fires on, its tier, and how it's actually captured — mechanism named specifically, not "the platform." Nothing else: no tracking-capability matrix, no legal analysis, no vendor comparisons. Those did their job getting here; this page is the result.

1. Enrichment signals — the discover → scrape → monitor pipeline

Same three-stage shape running for LinkedIn today. A fresh fact resolved by any stage is itself a signal — not just a data refresh.

PLATFORM	DISCOVER	SCRAPE	MONITOR	TIER
LinkedIn	ContactOut → Reverse Contact waterfall (tier 2 on a tier-1 miss, retried with name halves swapped). Union hit rate 63/100 vs. Clay's ~28-32/100 — why Clay got dropped. Fallback on a double-miss: Apify `harvestapi~linkedin-profile-search-by-name`, scored against the CRM, auto-accept or human review — lifts coverage to 78/100.	Apify `harvestapi~linkedin-profile-scraper`, $4/1k. Replaced `dev_fusion` after it went down platform-wide with no vendor response.	**Event-triggered, not calendar-scheduled** — a career-change signal fires a fresh scrape of the stored URL, not a blind periodic re-check.	T1
Instagram	Pipl	Apify Instagram Scraper / Bright Data IG API	Apify Schedule + Scraper Output Monitor	T1
Facebook	Pipl	Apify Facebook Scraper / Bright Data FB API	Apify Schedule + Scraper Output Monitor	T1
YouTube	Influencer Club's own Creator API (docs.influencers.club, self-serve — no Clay dependency) — consistently confirmed across sources; Pipl unconfirmed, needs a direct check	Bright Data YouTube API	Same, once a channel ID exists	T2

TikTok	Influencer Club's own Creator API — confirmed direction; Apify's TikTok actor ruled out, wrong direction (profile→contact, not email→profile)	Apify TikTok Scraper / Bright Data TikTok API	Same, once a handle exists	T2

Neither TikTok nor YouTube are open reverse-lookup directories — even a working provider here has a partial hit rate, not universal coverage. Worth setting that expectation before build, not discovering it after.

Fires as a signal when: **a new high-value fact resolves** — a job/role change, a new bio detail, a new post/engagement pattern the monitor catches on a diff run. The event is the change, not the data point sitting there unchanged.

LinkedIn's monitor is event-triggered off its own career-change signal, not a blind cron. The same pattern applies once IG/FB/TikTok have their own live signals (§2) — a DM or comment webhook firing is itself a reason to re-scrape that profile, which is sharper than a calendar-scheduled diff and worth building toward once those signals are live, rather than defaulting straight to cron scheduling.

2. Owned-social engagement signals

Fires on our own accounts only. Every one gives a platform-scoped identity — a handle, PSID, or channel ID — not automatically a full contact; that join happens once something bridges it.

SIGNAL	MECHANISM	TIER	NOTE
DM received (Instagram / Facebook)	Meta Webhooks — real-time push	T1	Highest-intent owned-social signal.
Comment / mention (Instagram / Facebook)	Meta Webhooks — real-time push	T2	Same identity caveat as DMs.
Comment on a YouTube video	YouTube Data API — **polled,** no webhook exists	T2	Scheduled poller with a cursor, not push.
DM received (TikTok)	TikTok Business Messaging API — webhook, gated access via Business Center	T1	Confirm UAE isn't region-excluded before relying on this — TikTok shuts out EEA/Switzerland/UK entirely.

Comment on TikTok	`TikTok Business Messaging API`	T2	Comment-to-DM linking is currently limited to Vietnam/Thailand/Indonesia — don't assume broader coverage.

2.1 — High-level implementation

Same five-stage shape for all four platforms, mirroring the WhatsApp/Zoho webhook receivers already running in this codebase (`app/api/webhooks/*`) — new sources into a pattern that already exists, not new architecture.

STAGE	WHAT HAPPENS
1. Ingress	A webhook endpoint per connected account for Instagram/Facebook (Meta Webhooks, real-time push) and TikTok (Business Messaging API, once region access is confirmed). YouTube has no webhook, so this stage is a scheduled poller against `commentThreads.list` instead — the one platform where "ingress" means polling, not receiving.
2. Normalize	Every platform's raw payload maps into one common event shape: `platform`, `event_type` (dm / comment / mention), `sender_platform_id` (IGSID / PSID / TikTok user ID / YouTube channel ID), `sender_handle`, `text`, `timestamp`, and `source_event_id` — the platform's own message/comment ID, which is what makes stage 5 idempotent.
3. Identity-resolve	Look up `sender_platform_id` against a stored mapping. A match attaches the event to the existing contact. No match creates a provisional, platform-scoped contact — handle and ID only, no email/phone yet — that a later bridge (a form fill, a phone number given in the conversation) can upgrade to a full contact. This is the partial-identity limit from §2 made concrete: the record exists from the first message, it just isn't complete yet.
4. Extract meaning	Run the §3 schema against `text`. Skipped for events with no text (a follow, a like) — nothing to extract.
5. Fire the signal	Write one row, keyed by `source_event_id` so a webhook replay can't double-fire it. Carries the signal type, the tier, and the extraction output from stage 4 — not just "an event happened."

3. Turning raw content into meaning

The gap LinkedIn doesn't have: a profile scrape returns structured fields (job title, company, seniority) for free. Everything in §2 gives back free text — a DM, a comment, a bio. "DM received" firing as one flat T1 signal doesn't distinguish a buying question from "hi" or an emoji. This section is the layer that closes that gap: one shared extraction schema, applied per platform, with the quirks that actually differ.

3.1 — The shared schema

Same mechanism already proven twice in this codebase — `lib/scoring/eligibility.ts` and the voice-call-summary work both run unstructured input through an LLM call against a fixed schema (`generateObject`, not a chat completion) and get structured fields back. This is that pattern, pointed at social text instead of a call transcript.

```
{
  intent_category: "product_question" | "pricing_or_availability"
                 | "general_interest" | "complaint" | "spam_low_value" | "other",
  sentiment:       "positive" | "neutral" | "negative",
  explicit_signal: boolean,   // real buying/interest language, not just a greeting
  confidence:      0.0–1.0
}
```

This structured output — not the raw DM — is what should actually modulate the signal's weight. A DM classified `pricing_or_availability` with `explicit_signal: true` stays or strengthens T1. One classified `spam_low_value` shouldn't score at all — it's still logged, so §6's discovery pass can still see the pattern, but it shouldn't move a contact's readiness.

3.2 — Instagram

INPUT	EXTRACTION	PLATFORM-SPECIFIC NOTE
DM text	Full schema (§3.1)	Highest-signal input on this platform — treat as the primary case the schema was designed for.
Comment text	Full schema	Public and often short — distinguish a genuine question ("how much is this?") from generic engagement ("🔥🔥") before it's allowed to count as a signal at all.
Story reply	Full schema	Same shape as a DM.
Bio text	Separate schema: profession_hint, location_hint, external_link	Run once at discovery/monitor time, not per-fire — a bio is a profile fact, not an event. Often contains a Linktree-style link worth capturing.
Follower count, post frequency, engagement rate	No LLM — numeric threshold classification	Mechanical, same tiering shape as any other rules-based signal — define what "active, sizeable audience" means as cutoffs, not a model call.

3.3 — Facebook

INPUT	EXTRACTION	PLATFORM-SPECIFIC NOTE
DM (Messenger) text	Full schema	Identical shape to Instagram's DM case — same Meta infra underneath.
Comment text	Full schema	Same low-information caveat as Instagram comments.
Page "About" fields	Partially free — category and location often arrive as structured fields already	Only the free-text description needs the extraction call; the rest is a direct field read, cheaper than Instagram's bio case.
Reviews (Page reviews, if present)	Star rating: numeric, free. Review text: full schema	A source Instagram and TikTok don't have — a rating is a structured signal on its own before any text extraction happens.

3.4 — YouTube

INPUT	EXTRACTION	PLATFORM-SPECIFIC NOTE
Comment text	Full schema, **plus the video's title/topic injected as context**	The one platform where the comment alone is meaningless — "love this!!" needs to know which video it's on. No other platform's extraction call needs this.
Channel description	Same profession/niche extraction as a bio	Useful mainly for classifying the commenter if they're a creator themselves, not for buying intent.
DM	Not available	No DM channel exists on YouTube at all — see §2. Nothing to extract from what doesn't exist.
Subscriber count, view counts, comment frequency	No LLM — numeric	Subscriber count is aggregate-only per §2 — a channel-health number, never joinable to the commenter's identity.

3.5 — TikTok

INPUT	EXTRACTION	PLATFORM-SPECIFIC NOTE
DM text	Full schema	Pending the region/access confirmation already flagged in §2 — the extraction step is moot if the DM channel itself isn't live here.
Comment text	Full schema, with a lower bar for `spam_low_value`	The noisiest input in this whole table — heavy on emoji and slang, low information density. Bias the classifier toward `spam_low_value` rather than risk false-positive intent detection.
Bio text	Same schema as Instagram's, lower expected yield	TikTok's ~80-character bio limit means there's usually just a link and an emoji — worth attempting, shouldn't be relied on the way Instagram's bio is.
Follower count, total likes, avg. video views, posting frequency	No LLM — numeric	Same mechanical tiering as Instagram's.

Cost note, not a footnote: this adds one LLM call per DM or comment that reaches the extraction step — the same per-call cost shape already established for the agentic layer elsewhere in this work. Cheap per call, but it scales directly with DM/comment volume once these channels are actually live, not something to assume is free because each call is small.

Sources: this repo's `DEPLOY_CHECKLIST.md` and prior review for the diagnosis behind this catalog's design · Meta Webhooks/Graph API docs, YouTube Data API docs, TikTok Business Messaging API docs, pipl.com/apify.com/brightdata.com/influencers.club product pages, and Apify's own actor input-schema documentation (checked directly, not inferred) for §1's YouTube/TikTok discovery evaluation · this repo's own `lib/ingest/providers.ts`, `lib/ingest/linkedin-finder.ts`, `lib/ingest/apify.ts`, `lib/signals/actions/trigger_re_enrichment_apify.ts`, and `lib/scoring/eligibility.ts`, read directly, for the LinkedIn row and the §3 extraction pattern.